

ADO.NET – introduction and applications

- ADO.NET is a part of the Microsoft .Net Framework.
- The full form of ADO.Net is ActiveX® Data Objects.
- ADO.Net has the ability to separate data access mechanisms, data manipulation mechanisms and data connectivity mechanisms.
- ADO.Net is a set of classes that allow application to read and write information in databases.
- ADO.Net can be used by any .Net Language.
- It's concept. It's not a programming language.
- ADO.Net introduces the concept of disconnected architecture.
- To work with ADO.Net user needs to add System.Data namespace.
- ADO .NET is a next version of ActiveX Data Objects (ADO) technology which was used in VB6.0.

Comparison between ADO and ADO.NET

Features	ADO	ADO.NET
Data access	ADO used connected data usage. (Connection-Oriented Models)	ADO.NET used disconnected data environment. (Disconnected Models)
XML Support	In ADO XML Support is limited.	In ADO.NET XML robust Support.

Format of data transferring	ADO used technology to access data and is COM – Based.	ADO.NET uses xml for transmitting data to and from your database and web application.
Data provider	In ADO disconnected data provide by Record Set.	In ADO.NET disconnected data provide by DataSet and DataAdpter.
Tables	In ADO, Record Set, is like a single table or query result.	In ADO.NET DataSet, can contain multiple tables.
Client connection	In ADO client connection model is very poor. Client application needs to be connected to data-sever while working on the data.	In ADO.NET client disconnected as soon as the data is fetched or processed. DataSet is always disconnected.

Advantages of ADO.NET

- 1) Interoperability
- 2) Maintainability
- 3) Programmability

4) Performance

5) Scalability

ADO.NET Architecture

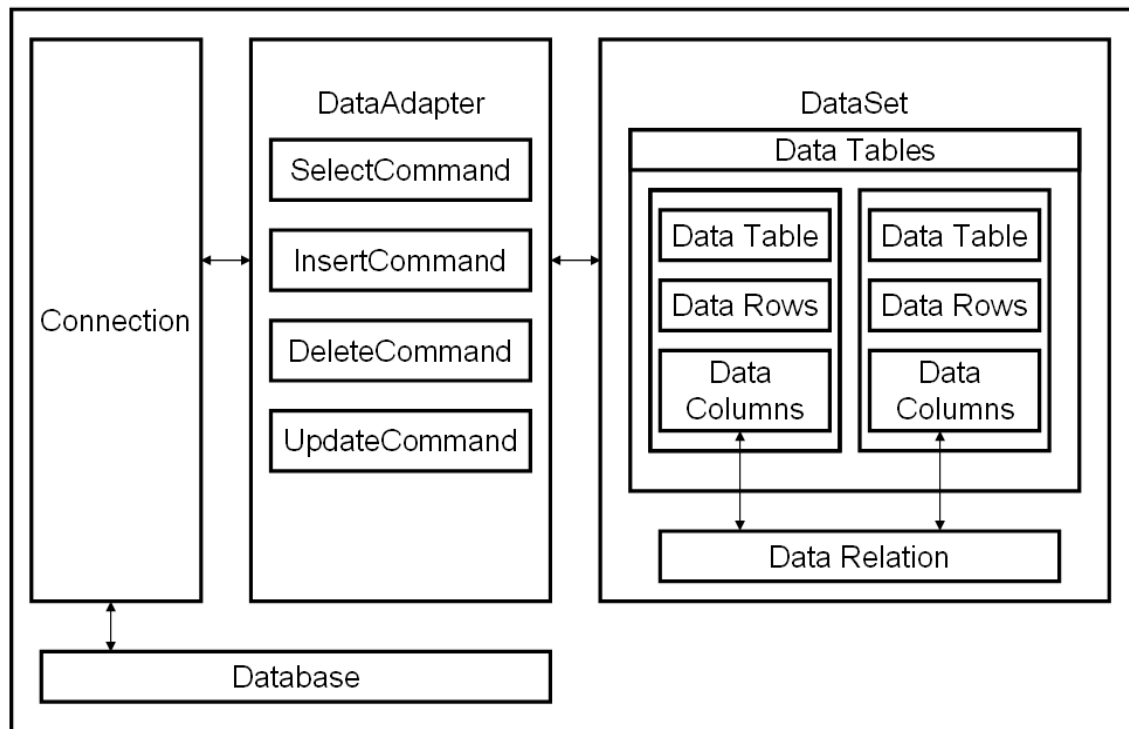


Figure 1: ADO .NET Architecture

The two main components of ADO.NET 3.0 for accessing and manipulating data are

- 1) The .NET Framework data providers
- 2) The DataSet.

1) .NET Framework Data Providers

- The .NET Framework Data Providers are components that have been explicitly designed for data manipulation and fast, forward-only, read-only access to data.

- Data Provider is a collection of components like Connection, Command, DataReader, DataAdapter objects
- The **Connection** object provides connectivity to a data source.
- The **Command** object enables access to database commands to return data, modify data, run stored procedures, and send or retrieve parameter information.
- The **DataReader** provides a high-performance stream of data from the data source.
- Finally, the **DataAdapter** provides the bridge between the **DataSet** object and the data source.
- The **DataAdapter** uses **Command** objects to execute SQL commands at the data source to both load the **DataSet** with data and reconcile changes that were made to the data in the **DataSet** back to the data source.

2) The DataSet

- The ADO.NET **DataSet** is explicitly designed for data access independent of any data source. As a result, it can be used with multiple and differing data sources, used with XML data, or used to manage data local to the application.
- The **DataSet** contains a collection of one or more **DataTable** objects consisting of rows and columns of data, and also primary key, foreign key, constraint, and relation information about the data in the **DataTable** objects.

Connected Architecture of ADO.NET

- The architecture of ADO.net, in which connection must be opened to access the data retrieved from database is called as connected architecture. Connected architecture was built on the connection, command, datareader and transaction.
- **Connection:** In connected architecture the purpose of connection is to just establish a connection to database and itself will not transfer any data.
- **DataReader:** DataReader is used to store the data retrieved by command object and make it available for .net application. Data in DataReader is read only and within the DataReader user can navigate only in forward direction.
- To access one by one record from the DataReader, **Read()** method is used which returns **bool** type data. When the next record was successfully read, the Read() method will return true otherwise returns false.

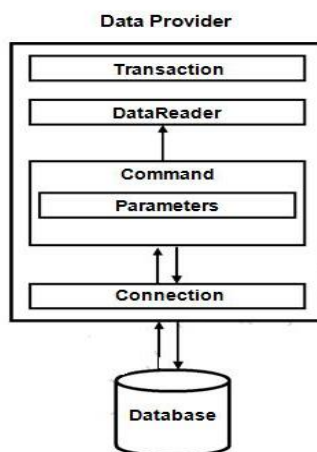


Figure:2: Connected Architecture

Disconnected Architecture in ADO.NET

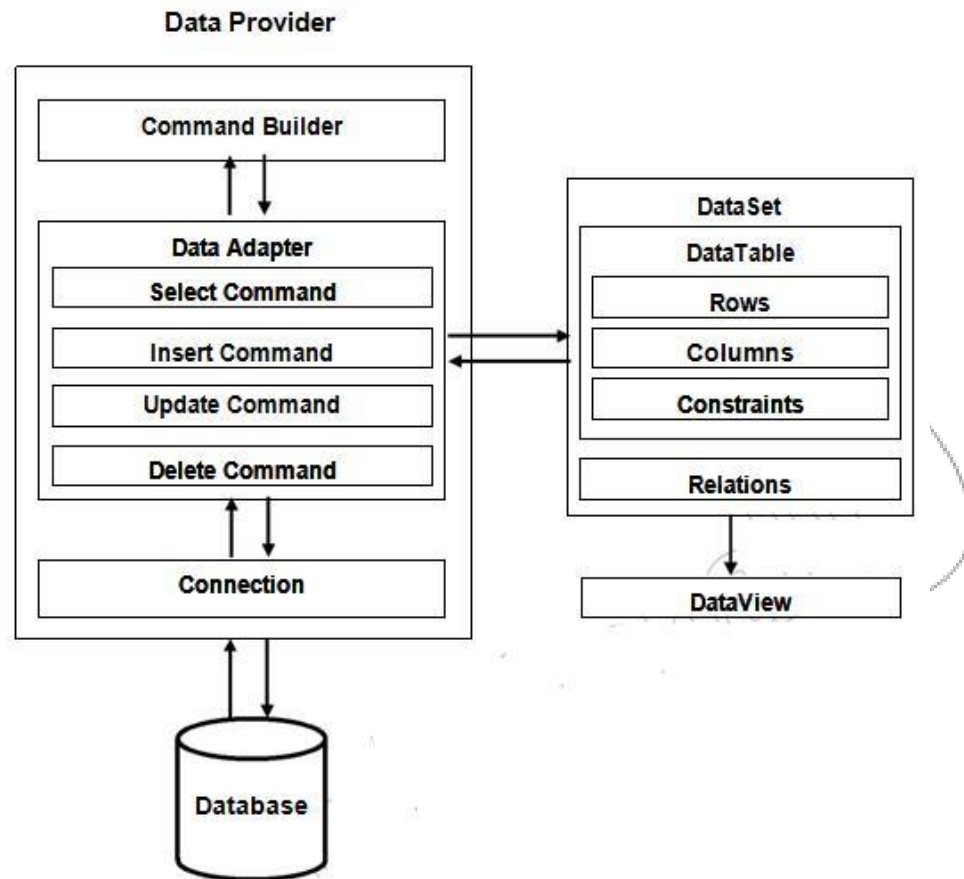


Figure:3: Disconnected Architecture

- The architecture of ADO.net in which data retrieved from database can be accessed even when connection to database was closed is called as disconnected architecture. Disconnected architecture of ADO.net was built on classes connection, dataadapter, commandbuilder and dataset and dataview.
- **Connection** : Connection object is used to establish a connection to database.

- **DataAdapter** : DataAdapter is used to transfer the data between database and dataset. It has commands like select, insert, update and delete. Select command is used to retrieve data from database and insert, update and delete commands are used to send changes to the data in dataset to database. It needs a connection to transfer the data.
- **Command Builder**: By default dataadapter contains only the select command and it doesn't contain insert, update and delete commands. To create insert, update and delete commands for the dataadapter, command builder is used. It is used only to create these commands for the dataadapter.
- **DataSet** : Dataset is used to store the data retrieved from database by dataadapter and make it available for .net application. A dataset can contain data in multiple tables. To fill data in to dataset **fill()** method of dataadapter is used and has the following syntax.

Da.Fill(Ds,"TableName")

- **DataView** : DataView is a view of table available in DataSet. It is used to find a record, sort the records and filter the records. By using dataview, you can also perform insert, update and delete as in case of a DataSet.

DataReader is **Connected Architecture** since it keeps the connection open until all rows are fetched one by one

DataSet is **DisConnected Architecture** since all the records are brought at once and there is no need to keep the connection alive

Difference between Connected and disconnected architecture

Connected (DataReader)	Disconnected (DataSet)
It is connection oriented.	It is disconnection oriented.
Connected methods gives faster performance	Disconnected get low in speed and performance.
connected can hold the data of single table	disconnected can hold multiple tables of data
connected you need to use a read only forward only data reader	disconnected you cannot
Data Reader can't persist the data	Data Set can persist the data
It is Read only, we can't update the data.	We can update data

ADO.NET Components

There are two major components of ADO.NET:

1. DataSet
2. DataProvider

DataSet : represents either an entire database or a subset of database. It can contain tables and relationships between those tables.

Data Provider is a collection of components like Connection, Command, DataReader, DataAdapter objects and handles communication with a physical data store and the dataset.

Data Provider

- The data provider is responsible for providing and maintaining the connection to the database.
- It is a set of classes that can be used for communicating with database, and holding/manipulating data
- The DataProvider connects to the data source on behalf of ADO.NET.
- The data source can be Microsoft SQL Server or Oracle database and OLEDB data provider.
- The data provider components are specific to data source.

The following lists the .NET Framework data providers that are included in the .NET Framework.

1) For SQL Server

- Imports System.Data.SqlClient namespace.
- It provides data access for Microsoft SQL Server.

2) For OLEDB

- Imports System.Data.OleDb namespace.
- It provides data sources exposed using OLEDB.
- User can use OLEDB to connect Microsoft Access.

3) For ODBC

- Imports System.Data.Odbc namespace.
- It provides data sources exposed using ODBC.

4) For Oracle

- Imports System.Data.OracleClient namespace.

- It provides data access for oracle.

Classes for Data Provider for all DataSource.

1. Connection
2. Command
3. DataAdapter
4. DataReader

1) Connction

- It establishes or connects a connection to the data source.
- In SQL Server the connection can establish using SqlConnection object.

2) Command

- Fires SQL commands or perform some action on the data source, such as insert, update, delete.
- In SQL Server command can fires using SqlCommand object.

3) DataAdapter

- It's a bridge between Data source and DataSet object for transferring data.
- In SQL Server the Data Adapter can create using SqlDataAdapter object

4) DataReader

- Used when large list of results one record at a time.
- It reads records in a read-only, forward-only mode.
- In SQL Server the datareader can create using SqlDataReader object

DataSet Object

- The **DataSet** is a major component of the ADO.NET architecture.
- Datasets are objects that contain data tables where user can temporarily store the data for use in your application.
- ADO.NET caches data locally on the client and store that data into DataSet.
- The dataset is a disconnected, in memory representation of data.
- It's not exact copy the database.
- It can be considered as a local copy of the some portions of the database.
- The DataSet contains a collection of one or more DataTable objects made up of rows and columns of data.
- Tables can be identified in DataSet using DataSet's Tables property.
- It also contains tables, rows, columns, primary key, foreign key, constraints and relation information about the data in the DataTable objects.
- DataSet are also fully XML-featured.
- Whatever operations are made by the user it is stored temporary in the DataSet, when the use of this DataSet is finished, changes can be made back to the central database for updating.
- The DataSet is populated DataAdapter's Fill method.

Fill Method: This method adds or refreshes rows in the DataSet.

Syntax: `instance.Fill(dataSet)`

Properties

Name	Description
------	-------------

Relations	Get the collection of relations that link tables and allow navigation from parent tables to child tables.
-----------	---

Tables	Gets the collection of tables contained in the DataSet.
--------	---

Update Method: It Calls the respective INSERT, UPDATE, or DELETE statements for each inserted, updated, or deleted row in the specified Dataset.

Syntax: `instance.Update(dataSet)`

The DataSet used to update the data source.

For Example :

```
Dim da As SqlDataAdapter
Dim ds As New DataSet
str1 = "select * from stud"
da = New SqlDataAdapter(str1, con)
da.Fill(ds)
DataGridView1.DataSource = ds.Tables(0)
```

Use of Server Explorer

- **Server Explorer/Database Explorer** is the server management console for Visual Studio. This window is used to open data

connections and to log on to servers and explore their system services.

- Use **Server Explorer/Database Explorer** to view and retrieve information from all of the databases you are connected to.
- From **View** menu user can access **Server Explorer/Database Explorer**.
- To make the window close automatically when not in use, select **Auto Hide** on the **Window** menu.
- User can do the following operations using Server Explorer:
 - List database tables, views, stored procedures, and functions
 - Expand individual tables to list their columns and triggers
 - Right-click a table to perform actions, such as showing the table's data or viewing the table's definition, from its shortcut menu.
- There are two primary nodes in the window.
 - 1) **Data Connections:** Top-level node that lists any data connections added by a user to the server. Beneath this node are listed the diagrams, tables, views, stored procedures, and functions for each connection. User can make data connections to many types of databases, including Oracle databases running on UNIX or Microsoft Windows operating systems.
 - 2) **Servers:** Top-level node that lists the servers currently available for use. Beneath each server node are listed the server resources you can use in your applications. User can drag nodes from the **Servers** node onto Visual Studio designers.

Namespace

System.Data	➤ The System.Data namespace provides access to classes that represent the ADO.NET architecture. ➤ Consists of the classes that constitute the ADO.NET architecture, which is the primary data access method for managed applications. ➤ The ADO.NET architecture enables user to build components that efficiently manage data from multiple data sources. ➤ ADO.NET also provides the tools to request, update, and reconcile data in distributed applications.
System.Data.SqlClient	➤ The System.Data.SqlClient namespace is the .NET Framework Data Provider for SQL Server.
System.Data.OleDb	➤ The System.Data.OleDb namespace is the .NET Framework Data Provider for OLE DB. ➤ Classes that make up the .NET Framework Data Provider for OLE DB-compatible data sources. ➤ These classes allow you to connect to an OLE DB data source, execute commands against the source, and read the results.

Connection Object

- A primary function of any database application is connecting to a data source and retrieving the data that it contains.
- In ADO.NET user can use a **Connection** object to create connection to the database.
- The connection object represents the physical connection to a data source.
- The Connection object contains all the information required to open a connection to the database. The .NET Framework Data Provider for SQL Server includes a SqlConnection object.

Steps to connect database :

1. Open new Project. From Solution Explorer Add Service Based Database(Extension .mdf)
2. From Server Explorer select the database, right click on it and select Add new table. So create and save the table.
3. From Server Explorer select the database and from properties window select the connectionstring and copy it. Then insert it into Connectionstring variable in the Form's Load event.
4. From Solution Explorer select the database and from properties window select FullPath and copy it in "app.config" file within connectionstring tag instead of "|directory|".
5. Also insert System.Data.SqlClient namespace at the top level of the current application.

- Connection object have `ConnectionString` property. Depends on the parameter specified in the Connection String, ADO.Net Connection Object connect to the specified Database.

Properties :

ConnectionString : Gets/sets the connection string to open a database.

Database : Gets the name of the database to open

DataSource : Gets the name of the SQL Server to use.

State : Gets the connection's current state

Methods :

Close : Closes the connection to the data provider.

Open : Opens a database connection.

ConnectionString: Gets or sets the string used to open a SQL Server database.

Syntax: instance.`ConnectionString` = *value*

Command Object :

- It's depended on Connection Object.
- Command objects are used to execute commands to a database across a data connection.

- The Command object in ADO.NET executes SQL statements and stored procedures against the data source specified in the connection object.
- The Command objects has a property called Command Text, which contains a String (Query) value that represents the command that will be executed in the Data Source.

For Example

```
Dim con As New SqlConnection
Dim cmd As SqlCommand
Dim str1 As String
Str1 = "Insert into stud values('"+ TextBox1.Text +"', '"+ TextBox1.Text
+''')
con.Open()
cmd = New SqlCommand(str1, con)
cmd.ExecuteNonQuery()
con.close()
```

OR

```
Dim con As New SqlConnection
Dim cmd As SqlCommand
con.Open()
cmd.Connection = con
cmd.CommandText = "Insert into stud values('"+ TextBox1.Text +"', '"+
TextBox1.Text +''')
cmd.ExecuteNonQuery()
```

con.close()

Properties

Property	Meaning
CommandText	Gets/sets the SQL statement (or stored procedure) for this command to execute.
CommandType	Gets/sets the type of the CommandText property (typically set to text for SQL).
Connection	Gets/sets the SqlConnection to use.
Parameters	Gets the command parameters.

Methods

Methods	Meaning
ExecuteNonQuery	Executes a non-row returning SQL statement, returning the number of affected rows.
	For Update, Insert and Delete statements this method is used with command object.
ExecuteReader	Returns Data Reader.
	Executes commands that return rows. For Select statement this method is used with command object.
ExecuteScalar	Executes the command and returns the value in the first column in the first row of the result.

	Returns single value.
	Used with aggregate functions.

DataAdapter Object :

- Data adapters are an integral part of ADO.NET managed providers, which are the set of objects used to communicate between a data source and a dataset.
- It is the bridge between the Dataset and Database.
- Adapters are used to exchange data between a data source and a dataset.
- In many applications, this means reading data from a database into a dataset, and then writing changed data from the dataset back to the database. However, a data adapter can move data between any source and a dataset.
- **SqlDataAdapter:** Represents a set of data commands and a database connection that are used to fill the Dataset and update a SQL Server database.
- When the **SqlDataAdapter** fills a DataSet, it creates the necessary tables and columns for the returned data if they do not already exist.
- **SqlDataAdapter** is used in conjunction with SqlConnection and SqlCommand to increase performance when connecting to a SQL Server database.

- The **SqlDataAdapter** also includes the **SelectCommand**, **InsertCommand**, **DeleteCommand**, **UpdateCommand** properties to facilitate the loading and updating of data.

Properties

Name	Meaning
DeleteCommand	Gets or sets a Transact-SQL statement or stored procedure to delete records from the data set.
InsertCommand	Gets or sets a Transact-SQL statement or stored procedure to insert new records into the data source.
SelectCommand	Gets or sets a Transact-SQL statement or stored procedure used to select records in the data source.
TableMappings	Gets a collection that provides the master mapping between a source table and a DataTable. (Inherited from DataAdapter.)
UpdateCommand	Gets or sets a Transact-SQL statement or stored procedure used to update records in the data source.

Methods	Meaning
Fill	Adds or updates rows in a data set to match those in the data source. Creates a table named "Table" by default

Update	Updates the data store by calling the INSERT, UPDATE, or DELETE statements for each inserted, updated, or deleted row in the given dataset.
--------	---

For Example :

```
Dim da As SqlDataAdapter
```

```
Dim ds As New DataSet
```

```
str1 = "select * from stud"
```

```
da = New SqlDataAdapter(str1, con)
```

```
da.Fill(ds)
```

DataReader Object

- User can use the ADO.NET **DataReader** to retrieve a read-only, forward-only stream of data from a database.
- User can not perform insert, delete and update operations in the DataReader.
- DataReader requires open connection.
- DataReader can hold one table at a time.
- DataReader supports read method to read the next record.
- DataReader can increase application performance both by retrieving data as soon as it is available, and (by default) storing only one row at a time in memory, reducing system overhead.

- Results are returned as the query executes, and are stored in the network buffer on the client until you request them using the **Read** method of the **DataReader**.
- .NET Framework Data Provider for SQL Server includes a **SqlDataReader** object.
- **SqlDataReader**: It provides a way of reading a forward-only stream of rows from a SQL Server database.

Properties

Name	Description
Connection	Gets the <u>SqlConnection</u> associated with the <u>SqlDataReader</u> .
IsClosed	Retrieves a Boolean value that indicates whether the specified <u>SqlDataReader</u> instance has been closed. (Overrides <u>DbDataReader.IsClosed</u> .)
Item	Overloaded. Gets the value of a column in its native format.

Methods

Name	Description
Close	Closes the <u>SqlDataReader</u> object. (Overrides <u>DbDataReader.Close()</u> .)
Read	Advances the <u>SqlDataReader</u> to the next record. (Overrides <u>DbDataReader.Read()</u> .)

Read Method: It advances the SqlDataReader to the next record.

Syntax: instance.Read()

This method returns **true** if there are more rows otherwise it returns **false**.

The default position of the SqlDataReader is before the first record.

Example:

```
Dim a1 As Integer, a2 As String, s1 As String
s1 = " "
con.Open()
str1 = "Select *from Emp"
cmd = New SqlCommand(str1, con)
Dim r1 As SqlDataReader
r1 = cmd.ExecuteReader()
While r1.Read()
    a1 = r1(0).ToString().Trim()
    a2 = r1(1).ToString().Trim()
    s1 = s1 & Space(3) & a1 & Space(3) & a2
    Me.lstPrint.Items.Add(s1)
    s1 = " "
End While
con.Close()
```

DataGridView Control

- The DataGridView control provides a powerful and flexible way to display data in a tabular format. User can use the DataGridView control to show read-only views of a small amount of data, or user can scale it to show editable views of very large sets of data.
- The DataGridView control provides a customizable table for displaying data. The DataGridView class allows customization of cells, rows, columns, and borders through the use of properties such as `DefaultCellStyle`, `ColumnHeadersDefaultCellStyle`, `CellBorderStyle`, and `GridColor`.
- User can use a DataGridView control to display data with or without an underlying data source. Without specifying a data source, you can create columns and rows that contain data and add them directly to the DataGridView using the `Rows` and `Columns` properties. User can also use the `Rows` collection to access `DataGridViewRow` objects and the `DataGridViewRow.Cells` property to read or write cell values directly. The `Item` indexer also provides direct access to cells.
- As an alternative to populating the control manually, user can set the `DataSource` and `DataMember` properties to bind the DataGridView to a data source and automatically populate it with data.
- When working with very large amounts of data, user can set the `VirtualMode` property to `true` to display a subset of the available data. Virtual mode requires the implementation of a data cache from which the DataGridView control is populated.

Explain the steps to bind DataGridView.

Step : 1	Take New VB.NET Project
Step : 2	Add database in your project (Named : dbemp.mdf) Add New Table in database file (Named : emp) from the Server Explorer Window Add some records in that table
Step : 3	Now add emp table to the DBEmpDataSet.xsd file
Step : 4	Add DataGridView control on the form
Step : 5	Select the Choose Data Source In that click on Other Data Source Project Data Source DBEmpDataSet Emp ← Table Name
Step : 6	At the end, Run the project

Binding Data Grids

To bind a data grid to a table, user can set the data grid's **DataSource** property (usually to a dataset, such as **dsDataSet**) and **DataMember** property (usually to text naming a table like "authors").

At run time, user can set both of these properties at once with the built-in data grid method **SetDataBinding** (data grids are the only controls that have this method):

```
DataGrid1.SetDataBinding(dsDataSet, "authors")
```

User can use these data sources with the data grid's DataSource property:

- DataTable objects
- DataView objects
- DataSet objects
- DataViewManager objects
- Single dimension arrays

Methods of Command Object:

Command object has 3 important methods to execute queries.

- 1) ExecuteNonQuery
- 2) ExecuteScaler
- 3) ExecuteReader

1) ExecuteNonQuery

- ExecuteNonQuery() is one of the most frequently used method in SqlCommand Object and is used for executing statements that do not return result set.
- ExecuteNonQuery() performs Data Definition tasks as well as Data Manipulation tasks also.

- The Data Definition tasks like creating Stored Procedures and Views perform by ExecuteNonQuery() .
- It is used to fire Data Manipulation tasks like Insert, Update and Delete queries to the database.

Example:

```
con.Open()  
str1 = "Delete from emp where eno='" +  
Me.txtEno.Text + "'" +  
cmd = New SqlCommand(str1, con)  
cmd.ExecuteNonQuery()  
MsgBox("Record is Deleted")  
con.Close()
```

2) ExecuteReader

- ExecuteReader() in SqlCommand Object send the SQL statements to Connection Object and populate a SqlDataReader Object based on the SQL statement.
- The SqlDataReader Object is a stream-based, forward-only, read-only retrieval of query results from the Data Source, which do not update the data.

Example:

```
Dim a1 As Integer, a2 As String, s1 As String  
s1 = " "  
con.Open()  
str1 = "Select *from Emp"
```

```
cmd = New SqlCommand(str1, con)
Dim r1 As SqlDataReader
r1 = cmd.ExecuteReader()
While r1.Read()
    a1 = r1(0).ToString().Trim()
    a2 = r1(1).ToString().Trim()
    s1 = s1 & Space(3) & a1 & Space(3) &
a2
    Me.lstPrint.Items.Add(s1)
    s1 = " "
End While
con.Close()
```

3) ExecuteScalar:

- ExecuteScalar() in SqlCommand Object is used for get a single value from Database after its execution.
- It executes SQL statements or Stored Procedure and returned a scalar value on first column of first row in the Result Set.
- If the Result Set contains more than one columns or rows, it takes only the first column of first row, all other values will ignore. If the Result Set is empty it will return a Null reference.
- It is used with aggregate functions like Count, Max, Min, Avg, Sum.
- It fetches the data from database and returns only one value at a time.

Example:

```
con.Open()
```

```
str1 = "select max(sno) from stud"  
cmd1 = New SqlCommand(str1, con)  
Dim i As Integer  
i = cmd1.ExecuteScalar()  
i = i + 1  
TextBox1.Text = i  
con.Close()
```

What are the four common SQL commands used to retrieve and modify data in a SQL Database? Also explain each of them.

Four Common SQL Commands :

1. Select
2. Insert
3. Update
4. Delete

Select: It retrieves data from one or more tables or views.

Syntax:

Select * from TableName

Select Columnname1, Columnname2 from TableName

Select * from TableName Where <Condition>

Select * from TableName Order by Columnname

Example:

Select * from stud

Select Sno,Sname from stud

Select * from stud where city = “Anand”

Select * from stud order by total

Insert: It creates a new row and inserts values into specified columns

Syntax:

Insert into TableName values(Value1,Value2,...)

Example :

Insert into stud values (1,'ABC')

Update: It changes the values of individual columns in one or more existing rows in a table.

Syntax :

Update TableName set Columnname = value,

Update TableName set Columnname = value where <condition>

Example :

Update stud set sname = 'xyz' where sno = 1

Delete: It deletes one or more rows from a table

Syntax:

Delete from TableName

Delete from TableName where <condition>

Example :

Delete from stud

Delete from stud where sno = 5

	Unit-4
	MCQ
1	_____ is disconnected, in-memory representation of data.
	a. DataReader b. DataSet
	c. DataAdapter d. DataCommand
2	_____ is the major component of connected data architecture.
	a. DataReader b. DataSet
	c. DataAdapter d. DataCommand
3	_____ method is used to populate DataSet.
	a. Populate b. Fill
	c. Open d. Store
4	_____ method returns result set by way of DataReader object.
	a. ExecuteDataReader b) ExecuteScalar
	c. ExecuteNonQuery d) ExecuteReader
5	For insert, update, and delete SQL commands, _____ method is used.
	a. ExecuteDataReader b) ExecuteScalar
	c. ExecuteNonQuery d) ExecuteReader
6	To check whether connection is open or not, _____ property is used.

	a. ConnectionStatus	b) State
	c. ConnectionState	d) Status
7	To use DataSet _____ namespace needs to be included.	
	a. System.Data.SqlClient	b) System.Sql
	c. System.Oledb	d) System.Data
8	_____ object provide connection to the database.	
	a. Command	b) Connection
	c. DataReader	d) DataAdapter
9	In a connection string, _____ represents name of the database.	
	a. DataSource	b) Initial Database
	c. Initial Catalog	d) Catalog Database
10	If we are not returning any records from the database _____ method is used.	
	a. ExecuteReader ()	b). ExecuteScalar ()
	c. ExecuteXmlReader()	d). ExecuteNonQuery()
11	Which namespace defines the interfaces implemented by a .NET Data Provider?	
	a. System.Data.	b. System.Data.OleDb and System.Data.SqlClient.
	c. System.Data.SqlTypes.	d. System.Data.Provider.
12	DataSet changes are saved to the database using the:	

	a. Save method.	b. Update method.
	c. Fill method.	d. None of the above.
13	DataSets are loaded from the database using the:	
	a. Load method.	b. Read method.
	c. Fill method.	d. None of the above.
14	In .NET, to work with directory and file _____ namespace is used.	
	a. System.Data	b. System.File
	c. System.IO	d. System.Dir
15	_____ method of command object is used with aggregate functions.	
	a. ExecuteReader ()	b. ExecuteScalar ()
	c. ExecuteXmlReader()	d. ExecuteNonQuery()

Short Questions		
1	What are the features of ADO.Net?	
2	Explain the use of server explorer in data access in .net.	
3	Mention the namespace that is used to include .NET Data Provider for SQL server in .NET code.	
4	Mention different types of data providers available in ADO .NET Framework.	
5	Which namespaces are required to enable the use of databases in ADO .net?	

6	What is the use of the Connection object?	
7	What are the usages of the Command object in ADO.NET?	
8	What is difference between DataSet and DataReader?	
9	What is Dataset object?	
10	What is connection string? Explain in brief.	
11	Which properties are used to bind a DataGridView control?	
12	Explain DataGrid control.	
14	Explain any one of following namespaces • System.Data • System.Data.SqlClient • System.Data.OleDb • System.Data.SqlTypes	
15	Explain any one of following <i>.NET Data Provider Main Classes</i> • Connection • Command • DataReader • DataAdapter	
16	List various methods of Directory class and explain any one of them.	
17	List various methods of File class and explain any one of them.	
Long Questions:--		
1	Explain the connected architecture of ADO.NET in brief.	[5]

2	Describe the disconnected architecture of ADO.NET's data access model.	[5]
3	Explain major ADO .net objects.	[5-10]
4	Explain the steps to bind the application with the Database in ADO .net.	[5]
5	Explain how can we retrieve data in DataSet?	[3-4]
6	Explain methods of Command object in detail.	[6-8]
7	What are the four common SQL commands used to retrieve and modify data in a SQL Database? Also explain each of them	[5-7]
8	Explain use of “ExecuteScaler”, “ExecuteNonQuery” and “ExecuteReader” methods in detail.	[6-8]